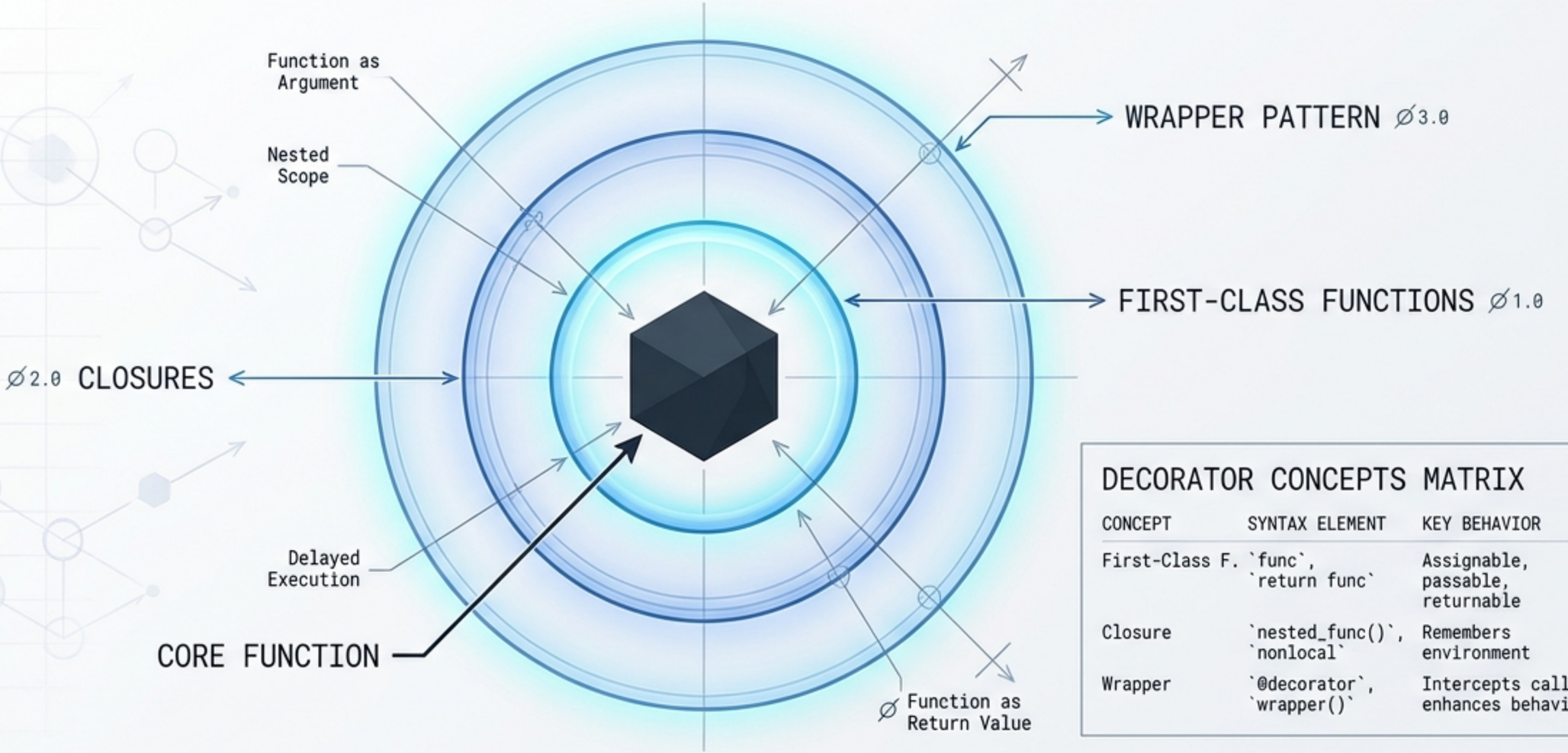
Demystifying Python Decorators

First-Class Functions, Closures, and the Wrapper Pattern



DECORATOR CONCEPTS MATRIX

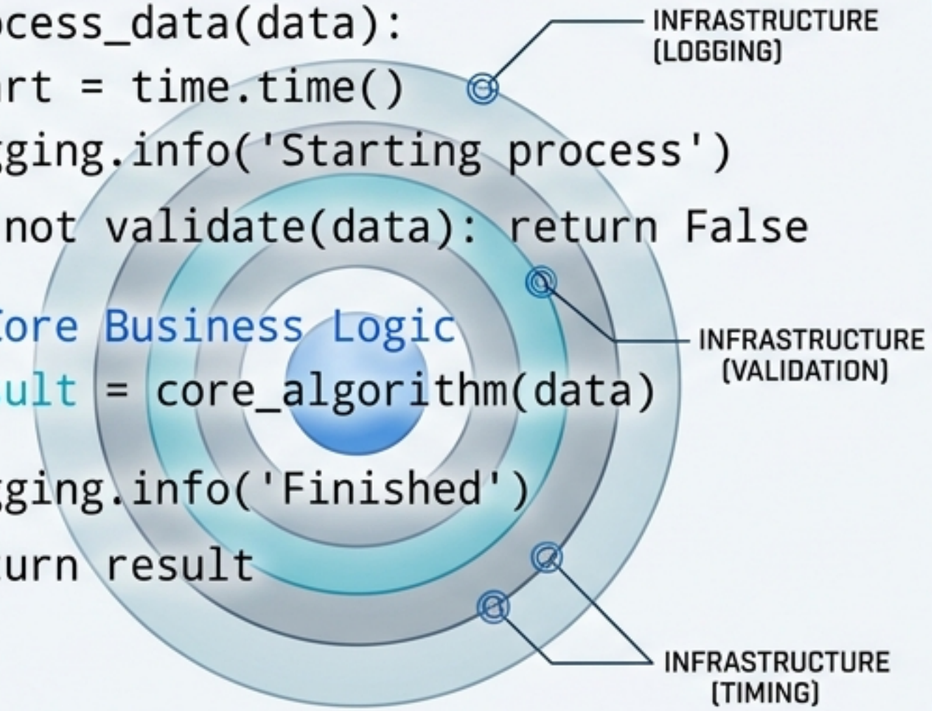
CONCEPT	SYNTAX ELEMENT	KEY BEHAVIOR
First-Class F.	`func`, `return func`	Assignable, passable, returnable
Closure	`nested_func()`, `nonlocal`	Remembers environment
Wrapper	`@decorator`, `wrapper()`	Intercepts calls, enhances behavior

Understanding the architectural blueprint of Python's most powerful design pattern.

BUSINESS LOGIC GETS BURIED UNDER INFRASTRUCTURE

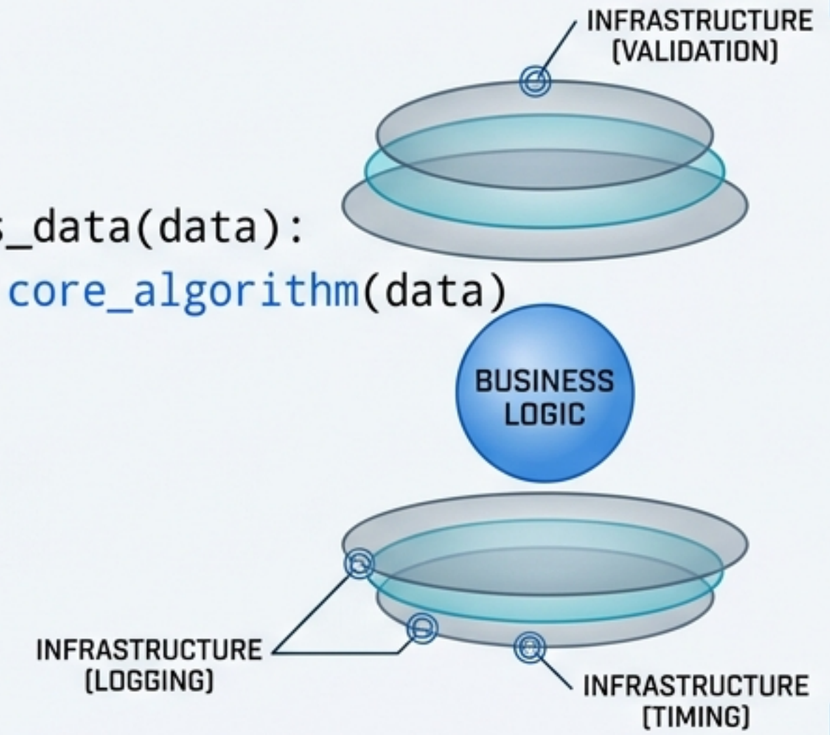
THE PROBLEM: TANGLED CONCERNS

```
def process_data(data):
    start = time.time()
    logging.info('Starting process')
    if not validate(data): return False

    # Core Business Logic
    result = core_algorithm(data)

    logging.info('Finished')
    return result
```



THE SOLUTION: SEPARATION OF CONCERNS

```
@timer
@logger
@validate
def process_data(data):
    return core_algorithm(data)
```



PRISTINE LOGIC

Business algorithms remain untouched and readable.

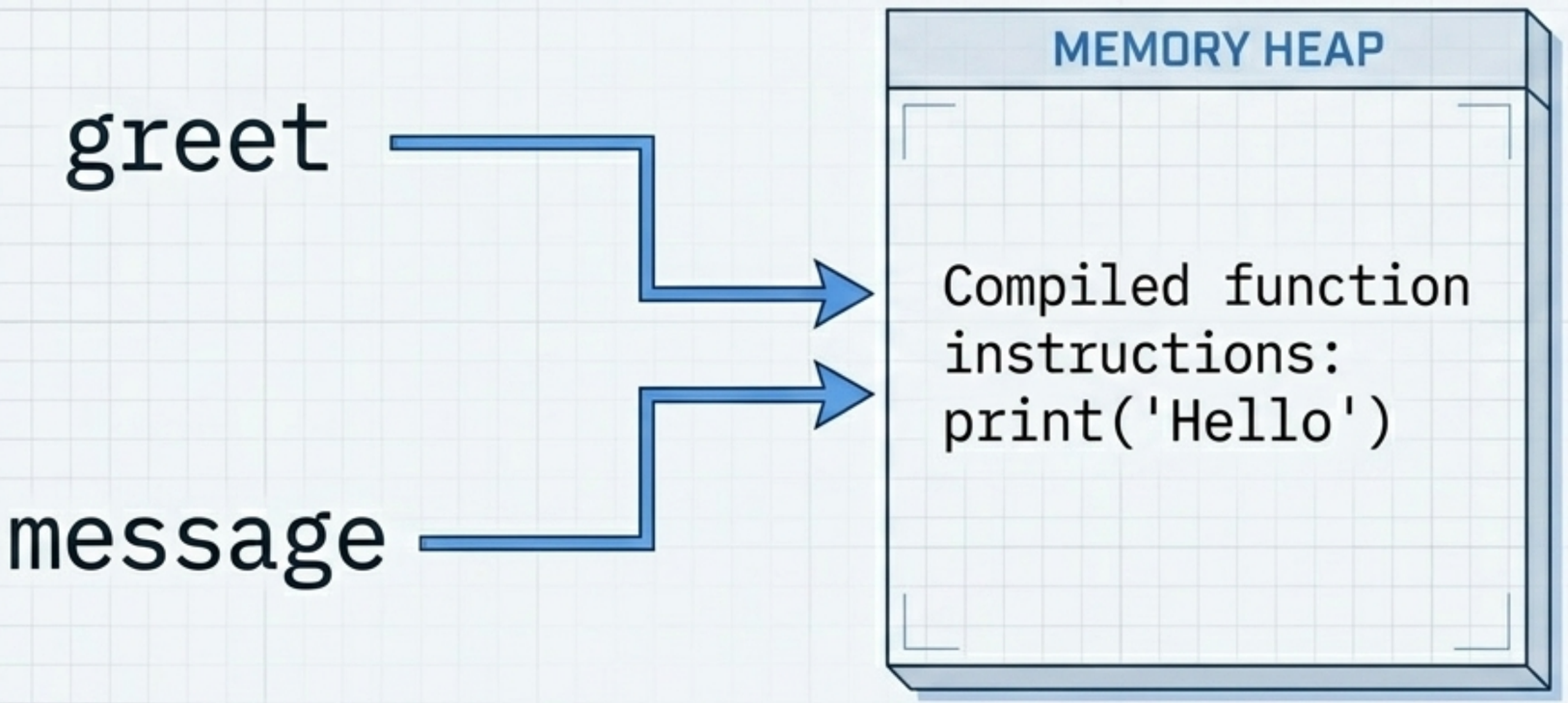
NO REGRESSION RISK

Edit infrastructure without breaking validated logic.

DRY PRINCIPLE

Apply the same infrastructure wrapper to any function.

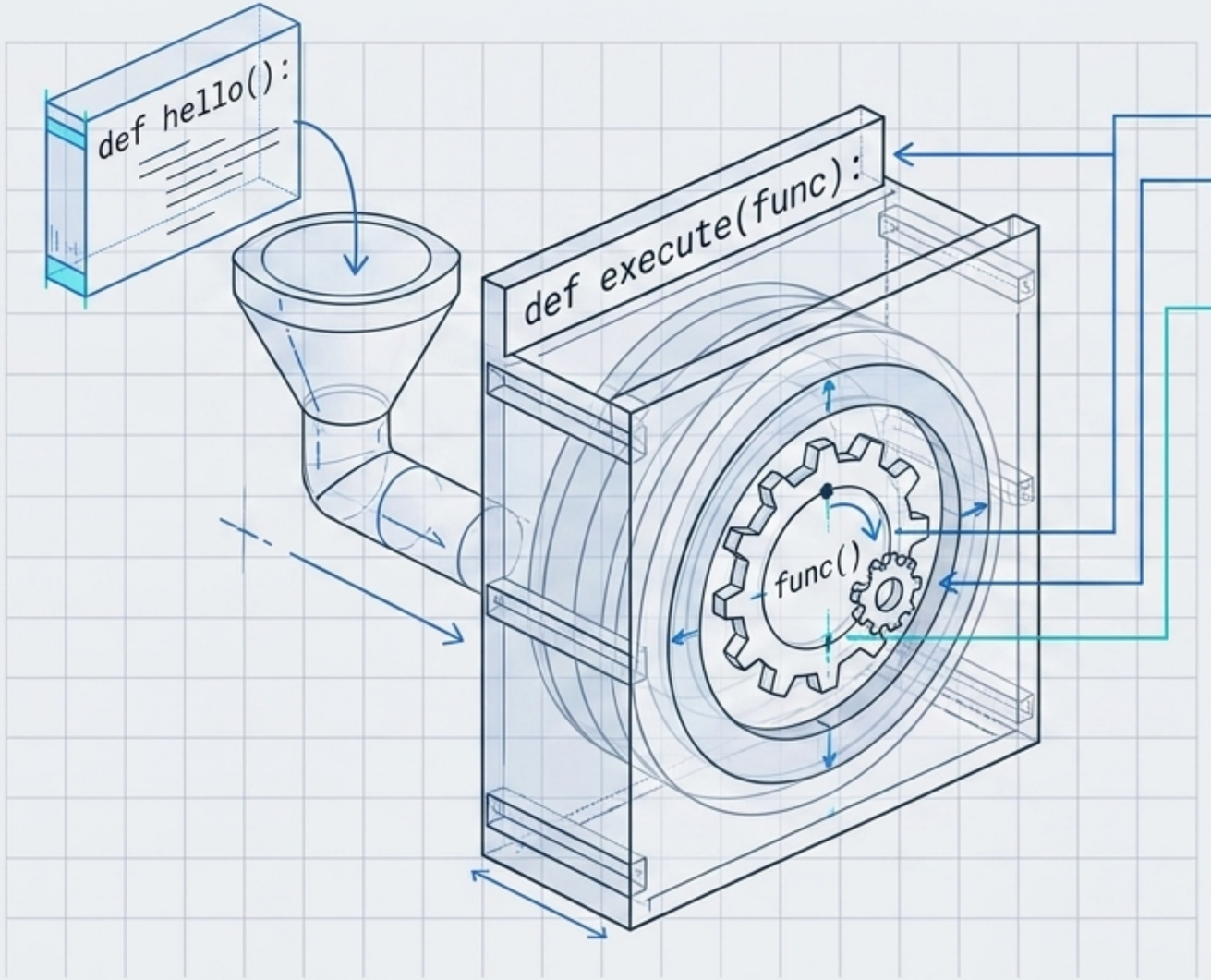
FOUNDATION 1:
FUNCTIONS ARE FIRST-CLASS OBJECTS



- Allocated on the heap, like integers or lists.
- Can be assigned to new variables without executing.
- The name is merely a reference, not the function's identity.

`message = greet` → `message()` executes `greet()`

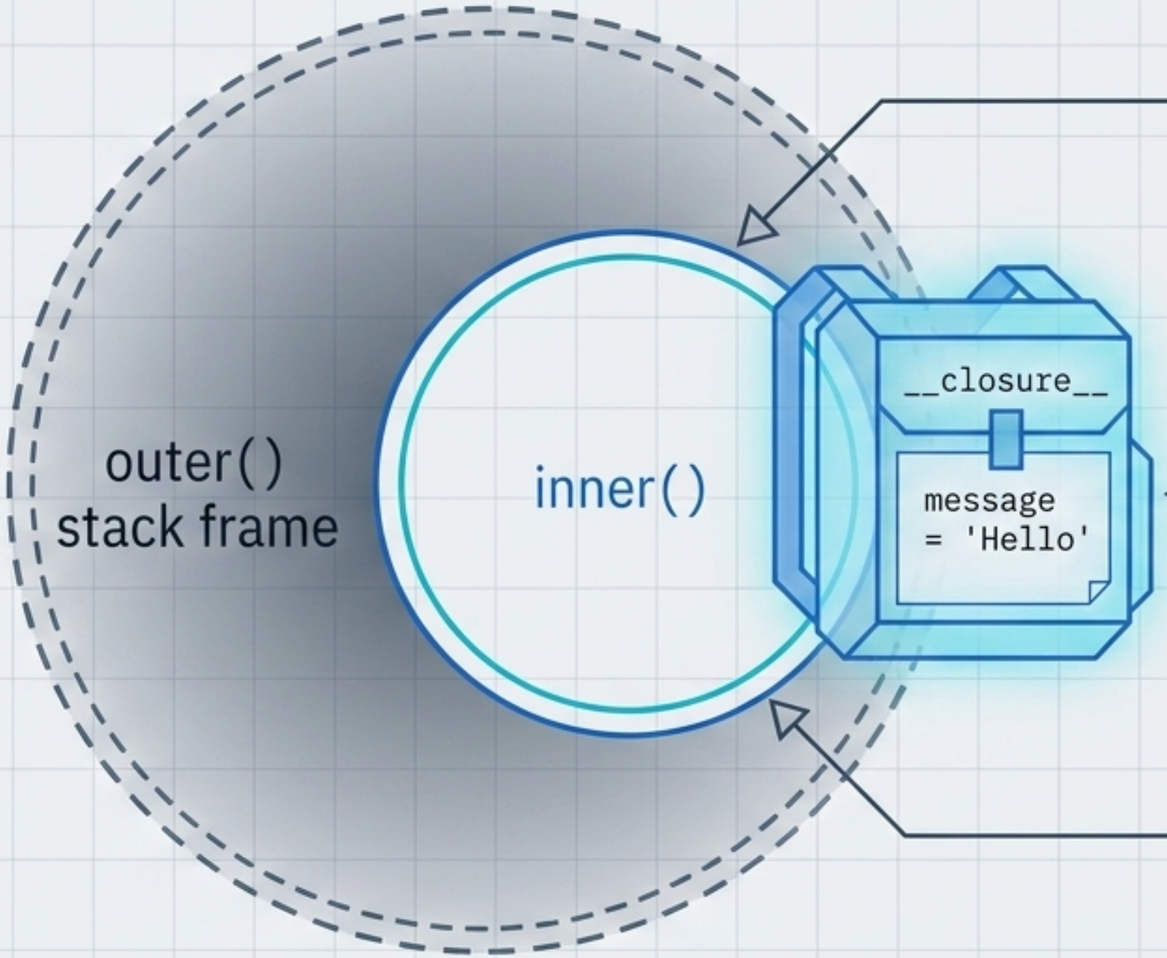
FOUNDATION 2: HIGHER-ORDER FUNCTIONS OPERATE ON FUNCTIONS




```
def execute(func):
    func()

execute(hello)
```

- Accept other functions as input arguments.
- Return new functions as resulting outputs.
- Allow callers to dynamically inject custom behaviour.

FOUNDATION 3: CLOSURES GIVE FUNCTIONS A MEMORY



Inner functions reference variables from enclosing scopes.

Variables survive even after the outer function returns.

This mechanism lets wrappers "remember" the original function.

THE SYNTAX REVEAL: DECORATORS ARE JUST SYNTACTIC SUGAR

```
@decorator
def greet(): ...
```

=

```
greet = decorator(greet)
```

DEFINITION TIME

Operates when the function is defined, not when it is called.

HIGHER-ORDER FUNCTION

Passes the original function into the decorator.

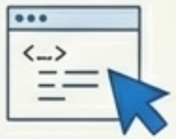
AUTOMATIC REBINDING

The original function name now references the new wrapper.

EXECUTION FLOW: DEFINITION TIME VS CALL TIME

PHASE 1: MODULE IMPORT (ONE-TIME COST)

FUNCTION CONSTRUCTED

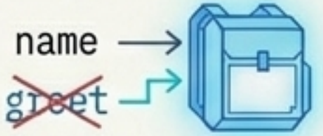


PASSED TO DECORATOR

```
def greet():
```

WRAPPER RETURNED

NAME REBOUND



PHASE 2: FUNCTION CALL (RECURRING COST)

WRAPPED NAME INVOKED

```
wrapped_function()
```

WRAPPER "BEFORE" LOGIC

```
print("Before")
```

ORIGINAL CORE FUNCTION

```
core_function()
```

WRAPPER "AFTER" LOGIC

```
print("After")
```

DECORATION IS A ONE-OFF STRUCTURAL ASSEMBLY. INVOCATION INCURS THE RECURRING WRAPPER OVERHEAD.

BUILDING A MINIMAL WRAPPER FROM FIRST PRINCIPLES

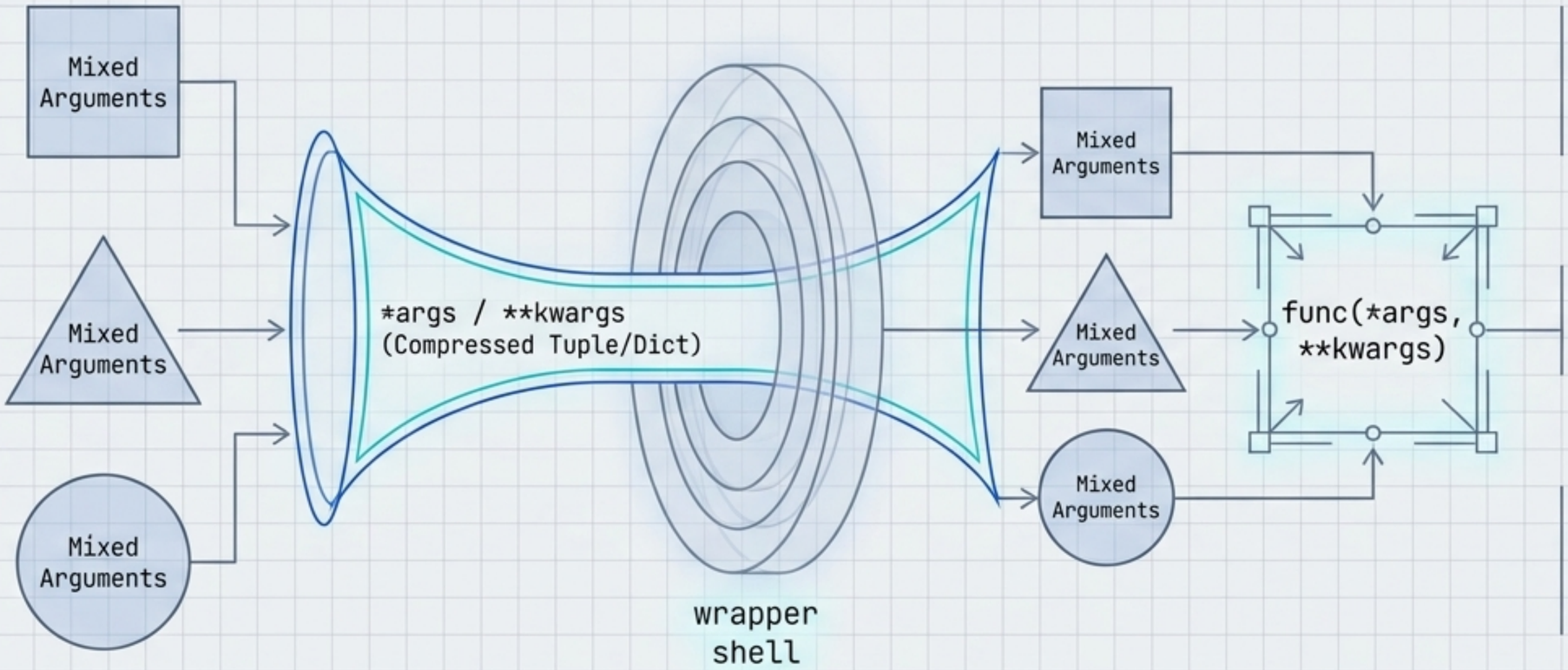
```
def email_decorator(func):
    def wrapper():
        print('Dear interns,')
        func()
        print('Best regards,')
    return wrapper
```

Outer Decorator
(Takes target function)

The Wrapper Shell
(Before/After side-effects)

The Core Call
(Delegates to original logic)

GENERALISING THE PATTERN FOR ARBITRARY SIGNATURES

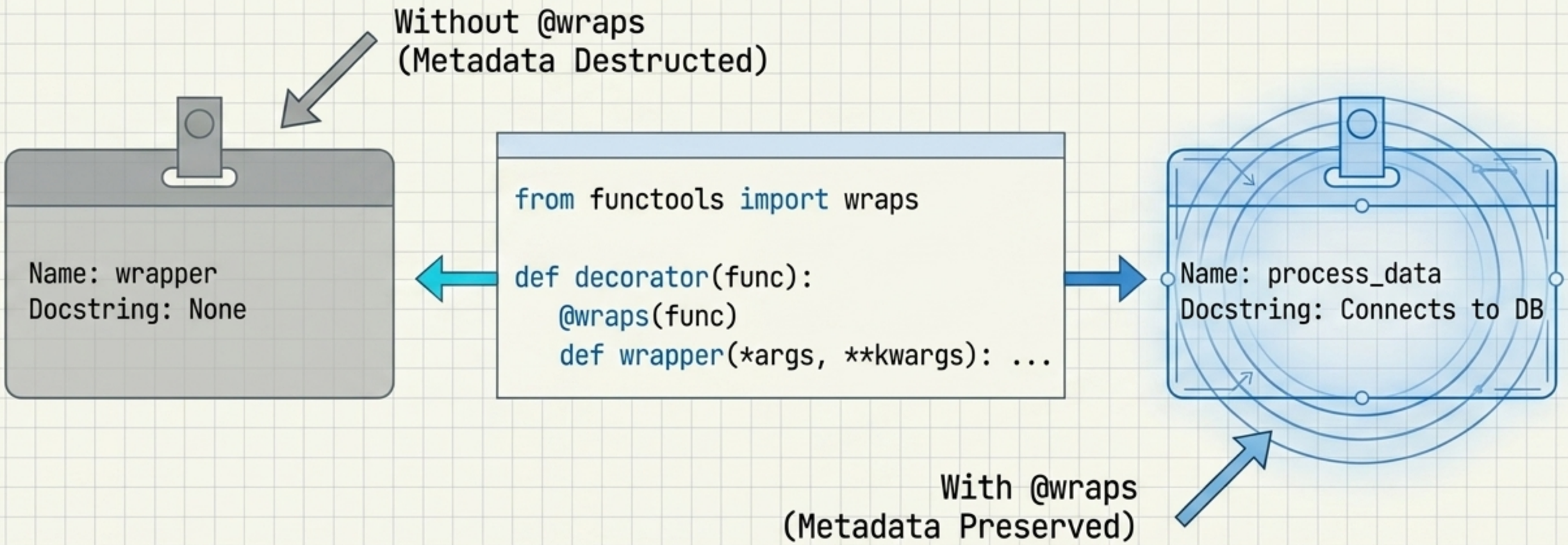


Zero-parameter wrappers break on complex functions.

Capture all inputs dynamically.

Forward identically to guarantee universal compatibility.

Preserving function identity with functools.wraps



A strict requirement for production-grade decorators to prevent breaking debuggers and auto-documenters.

The canonical decorator anatomy

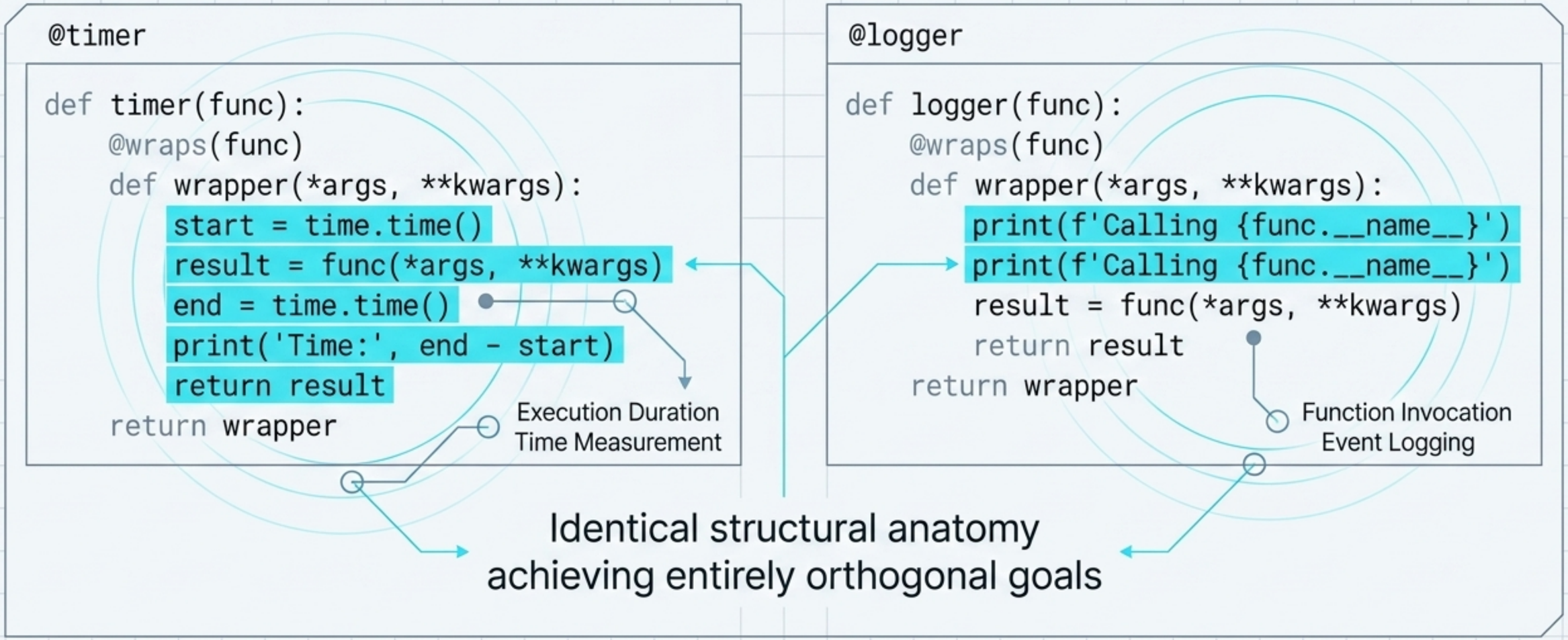
```python
def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # Pre-execution logic
        result = func(*args, **kwargs)
        # Post-execution logic
        return result
    return wrapper
```

1. Outer function capturing target
2. Metadata preservation
3. Arbitrary signature funnel
4. Core execution and capture
5. Return sequence

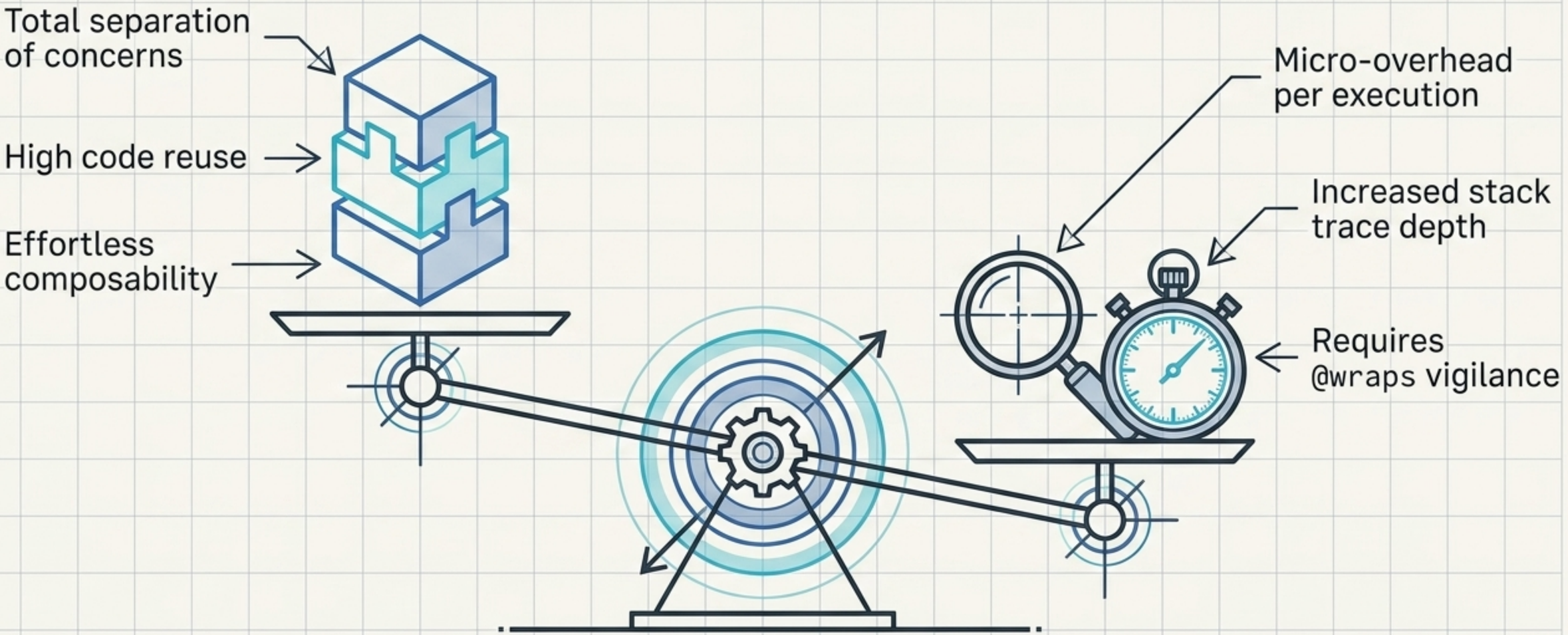
Standard library magic: Python's built-in decorators

Decorator	Mechanical Transformation	Primary Use Case
`@classmethod`	Modifies input: Receives `'cls'` instead of `'self'`	Factory patterns and alternative constructors
`@property`	Modifies interface: Accessed via `'obj.name'`	Computed attributes and clean encapsulation
`@lru_cache`	Modifies execution state: Intercepts calls via cache	Memoisation; converts exponential time to linear

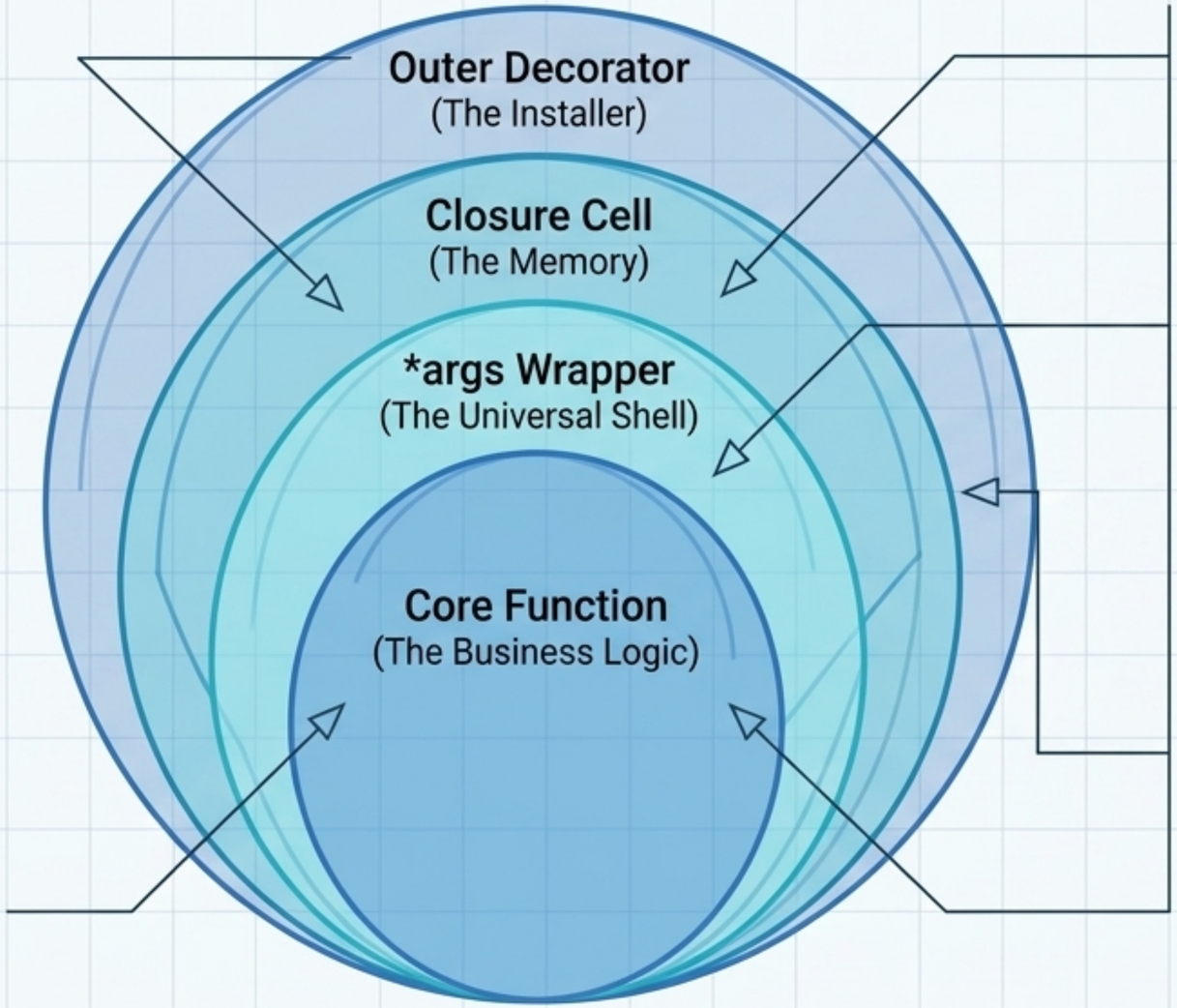
Real-world applications: Timing and Logging



Architectural benefits and runtime trade-offs



Summary: The blueprint assembled



Not magic: just higher-order functions leveraging closures.

Definition-time reassignment: `@decorator` **is simply** `func = decorator(func)`.

The idiomatic tool for clean, separable infrastructure logic.